

VS

1. ~~Clustering w/ k-means vs EM.~~
2. MLE vs MAP
3. Markov chain vs HMM
4. Fourier methods vs Autoencoder
5. MLP vs nonlinear regression
6. Bayesian vs MAP
7. RNN vs Kalman Filter
8. Naive Bayes vs SVM
9. Stable matching vs kuhn-munkres algorithm
10. Bayes classifier vs kNN
11. RTS smoothing vs Kalman Filter
12. KDE vs kNN vs K-means
 1. are all non-parameteric methods
13. MLE vs EM
14. maximizing likelihood vs minimizing entropy
15. conditional probability vs Bayes rule
16. word2vec vs glove
17. skipgram vs cbow
18. cross entropy vs kl divergence
19. `detach()` vs `requires_grad_(False)`
20. horn schunk vs lucas kanade
21. viterbi algorithm vs EM
22. precision recall curve vs ROC
23. ViT vs Dinov2
24. momentum vs RMSprop
25. multiscale conv vs multiscale feature extraction
26. ~~self-attention vs fully connected layer~~
27. bilinear interpolation vs transposed conv
28. viterbi vs baum-welch
29. Minimax vs MCTS
30. dp vs td vs mc
31. model parameters vs latent variables
 1. why this question? because they are similarly treated in variational inference.
32. value iteration vs policy iteration
 - 1.
33. generalization vs special case
34. Minimax vs Beam search [3]
 1. Beam search is similar to hill climbing algorithm, except we track multiple states simultaneously.
 1. Initialize: start with K random nodes
 1. Find all children of the K nodes.
 2. Add children and K nodes to pool, pick best
 3. repeat..
 2. unlike hill climbing, has more memory to better search hopeful options.
 3. Beam search with 3 beams.
 4. Pick best 3 options at each stage to expand.
 5. stopping criteria like hill-climb (when next pick is same as last pick).
 6. However the basic version of beam search can get stuck in local maximum as well.
 7. To help avoid this, stochastic beam search picks children with probability relative to their values.
 8. This is different that hill climbing algorithm with K restarts as better options get more consideration than worse ones.

2. The alternation of maximums and minimums is called minimax

1. Let's say you are threatening a friend to lunch. You choose either Shuang Cheng or Afro Deli.
2. The friend always orders the most inexpensive item, you want to treat your friend to best food.
3. Which restaurant should you go to?
4. Menus:
 1. Shuang Cheng: Fried rice=\$10.25, Lo Mein=\$8.55
 2. Afro Deli: Cheeseburger=\$6.25, Wrap=\$8.74
5. You could phrase this problem as a set of maximum and minimum as: $\max(\min(8.55, 10.25), \min(6.25, 8.55))$
6. which corresponds to $\max(\text{Shuang Cheng choices}, \text{Afro Deli choices})$
7. If our goal is to spend the most money on our friend, we should go to Shuang Cheng.
8. This representation works, but even in small games you can get a very large search tree.
9. For example in tic-tac-toe we have about 9! actions to search (or about 300,000 nodes).
10. larger problems (like chess or go) are not feasible for this approach (more on this next class).

35. Teacher forcing vs scheduled sampling

1. teacher forcing: use the ground truth as input instead of model output from prior time step.
2. for example: gt: "Sylvester Stallone has made some...", pred: "Sylvested Stallone is ...", instead using "is", we input "has" in the next time step.
3. advantages:
 1. faster, don't need to sample from output distribution
 2. model is not going to be good in the beginning, so it will output things that don't make any sense, without supervision the model will have a hard time learning.
4. issues:
 1. at inference we don't actually what the true token is
 2. the unknown prev token must be replaced by a token generated by the model itself.
 3. the discrepancy between teacher forcing during training and what we do during inference can yield errors that can accumulate quickly along the generated sequence.
5. Scheduled sampling: gently change the training process from a fully guided scheme using the previous token, towards a less guided scheme which mostly uses the generated token instead.

36. viterbi vs beam search [1]

1. Viterbi algorithm, Baum Welch algorithm, forward-backward algorithm, beam search, etc. are all very closely related, all using basically the same kind of dynamic programming. Viterbi calculates the max and argmax, Baum Welch (forward-backward) calculates the sum (forward) and then also the posteriors (backward). The difference is just the max vs sum. And to get the argmax, you would just backtrack. Beam search is an approximation for the max and argmax (bukan hajar kanan atau kanan mentok).

37. k-means vs DBSCAN vs GMM using EM

1. k-means fails in data shaped in concentric circles while DBSCAN could find the clusters in the concentric circles and handles outliers naturally.

38. viterbi vs dijkstra [2]

1. In the Viterbi algorithm, the total cost is a product, not a sum. You could change this into a sum by swapping all the incremental costs for their logarithms.
2. The Viterbi algorithm visits node in simple BFS order, so no priority queue is required. This works because the trellis is acyclic. You could visit then in Dijkstra order, but that would take longer in the worst case.

39. bag of freebies vs bag of specials

References:

1. <https://news.ycombinator.com/item?id=37886314>
2. <https://stackoverflow.com/questions/78647924/dijkstras-algorithm-vs-viterbi-algorithm>
3. <https://classpages.cselabs.umn.edu/Spring-2020/csci4511/slides/week6/02.26.20.pdf>
4. value iteration vs policy iteration <https://stackoverflow.com/a/37385098>, <https://ai.stackexchange.com/a/32018>
- 5.